JR Validated Environment

Administrator Manual: Statistics Test Suite

Document ID: JR-AM-002 | Version: 1.0 | Date: 2026-03-16

1. Purpose

This manual describes the statistics (OQ) test suite for the JR Validated Environment. It explains the folder structure, the administrator scripts used to run and validate the test environment, and how Python version and package pinning is managed. It is intended for the system administrator responsible for maintaining and running the OQ suite.

2. Overview

The OQ (Operational Qualification) suite verifies that all community scripts produce correct results under defined conditions. It runs as a standalone Python/pytest process in a dedicated virtual environment that is completely separate from the user analysis environment. This isolation ensures that test-runner packages (e.g. pytest) never contaminate the validated analysis environment.

The test suite is governed by the Statistics Validation Plan (docs/statistics_validation_plan.pdf, JR-VP-002 v1.0). Evidence of each run is written automatically to a timestamped text file that forms part of the validation record.

Note: The OQ suite tests the R and Python analysis scripts by calling them through jrrun and checking their output. It does not test the JR runtime infrastructure itself — that is covered by the IQ suite (admin_validate).

3. The oq/ Folder

All OQ test infrastructure lives in the oq/ subfolder at the project root. The key files are:

File / Folder	Purpose
oq/requirements.txt	Pins the exact pytest version used by the OQ venv.
oq/python_version.txt	Pins the Python version that the OQ venv must use.
oq/conftest.py	pytest configuration and shared fixtures.
oq/test_ss_suite.py	Sample size script tests.
oq/test_diagnostic_suite.py	Diagnostic script tests (normality, outliers, etc.).
oq/test_statistical_suite.py	Statistical analysis script tests (Bland-Altman, Weibull, etc.).
oq/test_gen_suite.py	Data generation script tests.
oq/test_convert_suite.py	Data conversion script tests.

The oq/data/ subfolder contains 12 synthetic CSV datasets used as test inputs. A .gitignore inside oq/data/ ensures the data files are tracked; pytest's own cache folder (.pytest_cache/) is excluded from version control.

4. OQ Virtual Environment

4.1 Location

The OQ venv is stored at a path derived from the project ID:

```
~/.venvs/<PROJECT_ID>_oq/
```

For a project with ID MyProject, this resolves to:

```
~/.venvs/MyProject_oq/
```

This location is separate from the user analysis venv (~/.venvs/MyProject/) and from the renv R library (~/.renv/MyProject/). All three environments are managed independently.

4.2 Automatic Management

admin_oq creates and rebuilds the OQ venv automatically. The venv is rebuilt if:

- The venv directory does not exist.
- The SHA256 hash of oq/requirements.txt has changed since the last build.

The hash is stored inside the venv at .oq_req_hash. Administrators do not need to manage the venv manually under normal circumstances.

Warning: Do not install packages into the OQ venv by hand. Any manual changes will be overwritten the next time admin_oq detects a requirements.txt change.

4.3 Manual Rebuild

To force a full venv rebuild:

```
rm -rf ~/.venvs/MyProject_oq/
./admin/admin_oq
```

admin_oq will detect the missing venv and rebuild it from oq/requirements.txt before running the tests.

5. oq/python_version.txt

This file pins the exact Python version that the OQ venv must use. It contains a single line with the version number and no trailing spaces:

```
3.11.9
```

5.1 Purpose

The pin ensures that the OQ tests are always run under a known, reproducible Python version. admin_oq_validate reads this file and compares it against the Python binary installed in the OQ venv, failing with a clear error if there is a mismatch.

5.2 Updating the Pin

The pin should only be updated deliberately, after confirming that the new Python version is available on all machines that will run admin_oq. To update:

- Edit oq/python_version.txt and replace the version number.
- Recreate the OQ venv with the new Python version (see Section 4.3).
- Run the full OQ suite (admin_oq) to confirm all tests still pass.
- Run admin_create_hash to regenerate the integrity file.
- Commit both oq/python_version.txt and admin/project_integrity.sha256.

Validation impact: Changing the Python version changes the test execution environment. This requires a new OQ run and review before the updated report can be signed off. Document the version change in the statistics_validation_report.

6. oq/requirements.txt

This file pins the Python packages installed in the OQ venv. It uses standard pip requirements format with exact version pins (==). The current content is:

```
pytest==8.3.4
```

6.1 Purpose

Using a pinned requirements file guarantees that the same pytest version is used on every machine and at every point in time. This is required for regulatory traceability — the OQ evidence header records the pytest version, and that version must match what was used when the validation report was signed.

6.2 Adding or Updating Packages

If additional pytest plugins or helper packages are needed:

- Add the package with an exact version pin to oq/requirements.txt.
- Delete the OQ venv so that admin_oq rebuilds it: rm -rf ~/.venvs/MyProject_oq/
- Run admin_oq_validate to confirm the new package is installed correctly.
- Run the full OQ suite.
- Run admin_create_hash and commit oq/requirements.txt.

Note: The two Python analysis scripts (jrc_convert_csv.py, jrc_convert_txt.py) use only the Python standard library. No additional packages are required for those scripts, and they do not affect oq/requirements.txt.

7. admin/admin_oq_validate

admin_oq_validate is a pre-flight checker that verifies the OQ environment before running admin_oq. It is run manually by the administrator and produces a clear pass/fail summary for each check.

7.1 When to Run

Run admin_oq_validate:

- Before the first admin_oq run on a new machine.
- After updating oq/python_version.txt or oq/requirements.txt.
- After recreating the OQ venv for any reason.
- Any time admin_oq produces unexpected errors related to the environment.

7.2 How to Run

```
./admin/admin_oq_validate
```

The script prints a section header and a pass/fail line for each check. If all checks pass it exits with code 0. If any check fails it exits with code 1 and prints remediation instructions.

7.3 Checks Performed

#	Check	What Is Verified
1	python_version.txt exists	oq/python_version.txt is present and contains a non-empty version string.
2	OQ venv exists	The directory ~/.venvs/<PROJECT_ID>_oq/ exists.
3	Python version matches pin	The Python binary in the OQ venv reports the exact version in oq/python_version.txt.
4	Packages installed at pinned versions	Every pinned package in oq/requirements.txt is installed at the exact required version.

7.4 Interpreting Output

Each check line starts with one of three symbols:

- ✅ Check passed — no action required.
- ❌ Check failed — follow the remediation instructions printed below the line.
- ⚠️ Warning — package has no version pin and was skipped.

When a check fails, admin_oq_validate prints the specific command needed to fix the issue (e.g. the pip install command or the venv recreation command). After applying the fix, re-run admin_oq_validate to confirm all checks pass before proceeding to admin_oq.

8. admin/admin_oq

admin_oq is the main OQ execution script. It manages the OQ venv, runs the full pytest suite, and writes a timestamped evidence file.

8.1 How to Run

```
./admin/admin_oq
```

To pass custom pytest arguments (e.g. run a single test file):

```
./admin/admin_oq -v oq/test_ss_suite.py
```

If no arguments are given, admin_oq defaults to -v --tb=short, which prints verbose test names and short tracebacks on failure.

8.2 What It Does

In order:

- Reads admin/project_id.txt to determine the project ID.
- Checks whether the OQ venv needs to be rebuilt (missing venv or changed requirements.txt hash).
- Creates the OQ venv and installs packages from oq/requirements.txt if a rebuild is needed.
- Creates the evidence output directory (~/.jrscript/<PROJECT_ID>/validation/) if it does not exist.
- Writes an evidence header to a new timestamped file (oq_execution_<YYYYMMDDTHHMMSS>.txt).
- Runs pytest and tees output to both the terminal and the evidence file.
- Appends an evidence footer recording OQ RESULT: PASS or FAIL and the completion timestamp.

8.3 Evidence Output

Evidence files are written to:

```
~/.jrscript/<PROJECT_ID>/validation/oq_execution_<timestamp>.txt
```

Each evidence file contains the complete pytest output plus a structured header and footer recording: project ID, timestamp, hostname, R version, Python version, pytest version, test directory, and the overall OQ RESULT. These files form part of the regulatory validation record and must be retained.

Note: Evidence files are stored outside the project repository (in ~/.jrscript/) so they are not subject to git history and can accumulate across multiple runs. Back them up according to your document control procedure.

8.4 Recommended Sequence

The recommended procedure before each formal OQ run:

Step	Command	Purpose
1	./admin/admin_create_hash	Regenerate integrity hash after any file changes.
2	./admin/admin_oq_validate	Confirm OQ environment is correctly configured.
3	./admin/admin_oq	Run the full OQ suite and write evidence.
4	Review evidence file	Confirm OQ RESULT: PASS and check for deviations.

Important: admin_create_hash must be run before admin_oq any time scripts, wrappers, help files, or requirements files have been added or changed. If the integrity hash is stale, jrrun will fail its integrity check and all OQ tests will fail.

9. Troubleshooting

Symptom	Resolution
All OQ tests fail immediately	Run admin_create_hash. A stale integrity hash causes jrrun to abort before any script runs.
admin_oq exits with code 1, all tests pass	Verify admin_oq uses pipestatus[1] (zsh, lowercase) not PIPESTATUS[0] (bash). See commit 0913142.
OQ venv not found	Run admin_oq — it will create the venv automatically. Or run admin_oq_validate for diagnostics.
Python version mismatch	Install the pinned Python version and recreate the venv, or update oq/python_version.txt if the installed version is acceptable. Re-run admin_oq and produce new evidence before signing off.
Package version mismatch	Activate the OQ venv and run: pip install -r oq/requirements.txt. Then re-run admin_oq_validate.